# Databricks Fundamentals

**Teoria, demonstracje i praktyka**

Sześć modułów podstawowych i dodatkowy AI/BI

# Agenda dnia

**09:00**
Wprowadzenie i setup

**09:15**
M01 · Platforma i compute

**10:15**
M02 · Apache Spark

**10:40**
M03 · Ingestia + warsztat 11:30

**12:40**
M04 · Delta Lake + warsztat 13:20

**14:05**
M05 · Lakeflow Jobs

**14:35**
M06 · Unity Catalog

**14:50**
Projekt · 15:50 podsumowanie

Przerwy: 10:30 i 13:55 · Lunch: 12:10–12:40 · Koniec: 16:00
M07 AI/BI: dodatkowy pokaz, jeśli zostanie czas.



# Krzysztof Burejza

## Resident Solution Architect @ Databricks

10 lat pracy z danymi: SQL Server, Azure, Databricks i Microsoft Fabric.

Buduję pipeline’y, hurtownie danych i lakehouse’y z zespołami klientów.

## Microsoft Certified Trainer

Prowadzę szkolenia i piszę na dataistheway.blog.

## Certyfikaty

Databricks Data Engineer Professional

Databricks Generative AI Engineer Associate

Azure AI Engineer Associate · Fabric Data Engineer Associate



# Poznajmy się

Przedstaw się w jednym zdaniu.
Z jakimi danymi i narzędziami pracujesz?
Czy znasz już Databricks?
Co chcesz umieć po tym szkoleniu?

# Dla kogo jest to szkolenie?

## Dla kogo?

Analitycy i inżynierowie danych

Specjaliści BI

Osoby pracujące z pipeline’ami

## Przyda Ci się

SQL: SELECT, JOIN, GROUP BY

Podstawy relacyjnych baz danych

## Zaczynamy od podstaw

Databricks i Spark poznasz na szkoleniu.

Python wprowadzimy w zakresie potrzebnym do PySpark.



altkom akademia

# Zasady szkolenia

Pytaj, eksperymentuj i sprawdzaj wynik.

### Pytaj od razu

Wyjaśnijmy pojęcie, zanim przejdziemy do kodu.

### Pracuj we własnym tempie

Najpierw zadania główne, potem OPTIONAL.

### Czytaj błędy

Sprawdź komunikat, dane i przyczynę.

### Współpracuj

Porównuj pomysły. Uruchamiaj kod we własnym katalogu.

### Weryfikuj kod AI

Przeczytaj kod i sprawdź wynik przed zapisem danych.

# Sześć modułów i dodatkowy AI/BI

**M01 · Platforma**
Workspace, compute, storage i koszty

**M02 · Apache Spark**
Driver, executory i wykonanie obliczeń

**M03 · Ingestia**
Pliki, schemat, transformacje i medalion

**M04 · Delta Lake**
Transakcje, MERGE i historia danych

**M05 · Lakeflow Jobs**
Zależności, parametry i harmonogram

**M06 · Unity Catalog**
Organizacja danych i uprawnienia

**DODATKOWO M07 · AI/BI: dashboardy i Genie**

# Od pojęcia do działającego rozwiązania

→ **1. Zrozum mechanizm**

Do czego służy komponent i jak działa?

→ **2. Zobacz demonstrację**

Te same pojęcia w workspace i notebooku.

→ **3. Wykonaj ćwiczenie**

SQL i PySpark. Instrukcje są w notebooku.

→ **4. Wyjaśnij wynik**

Co się zmieniło i dlaczego? Co oznacza błąd?

M01 / PLATFORMA

# Platforma Databricks

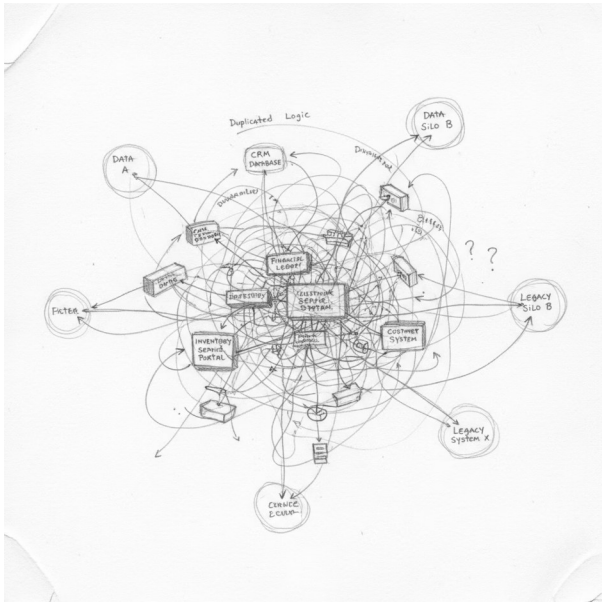
Pierwsze 30–45 minut: platforma, architektura i narzędzia.

# Hurtownia, data lake i lakehouse










## Hurtownia danych

Tabele i uzgodniony model

SQL i raporty biznesowe

## Data lake

Pliki w object storage

Różne formaty danych

## Lakehouse

Tabele nad plikami

Transakcje i kontrola dostępu

# Czym jest Databricks?

Jedna platforma dla danych, analityki i AI.

**Data engineering**

Wczytanie i przygotowanie danych

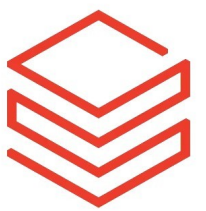
**SQL i BI**

Zapytania, raporty i dashboardy

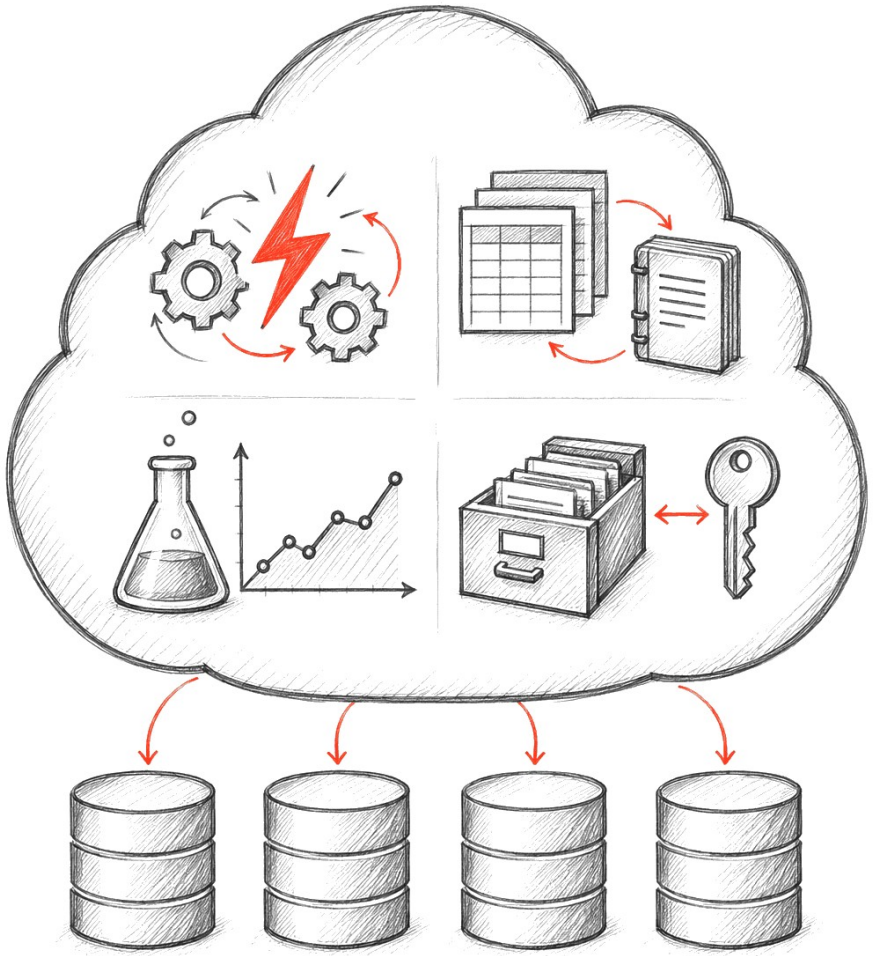
**Data science i AI**

Eksperymenty, modele i aplikacje

Ćwiczymy w Azure Databricks.



databricks



# Technologie i ich role

Każdy komponent rozwiązuje inny problem.

### Apache Spark

Przetwarza dane: SQL i DataFrame.

### Delta Lake

Format tabel: transakcje i historia.

### MLflow

Eksperymenty i modele AI. Poza ćwiczeniami.

### Unity Catalog

Metadane, dostęp i pochodzenie danych.

Na szkoleniu korzystamy z Azure Databricks.

# Kto zarządza compute?

## Control plane

Workspace i konfiguracja

Zarządzanie compute

Harmonogramy jobów

Usługi zarządzane przez Databricks

## Compute plane

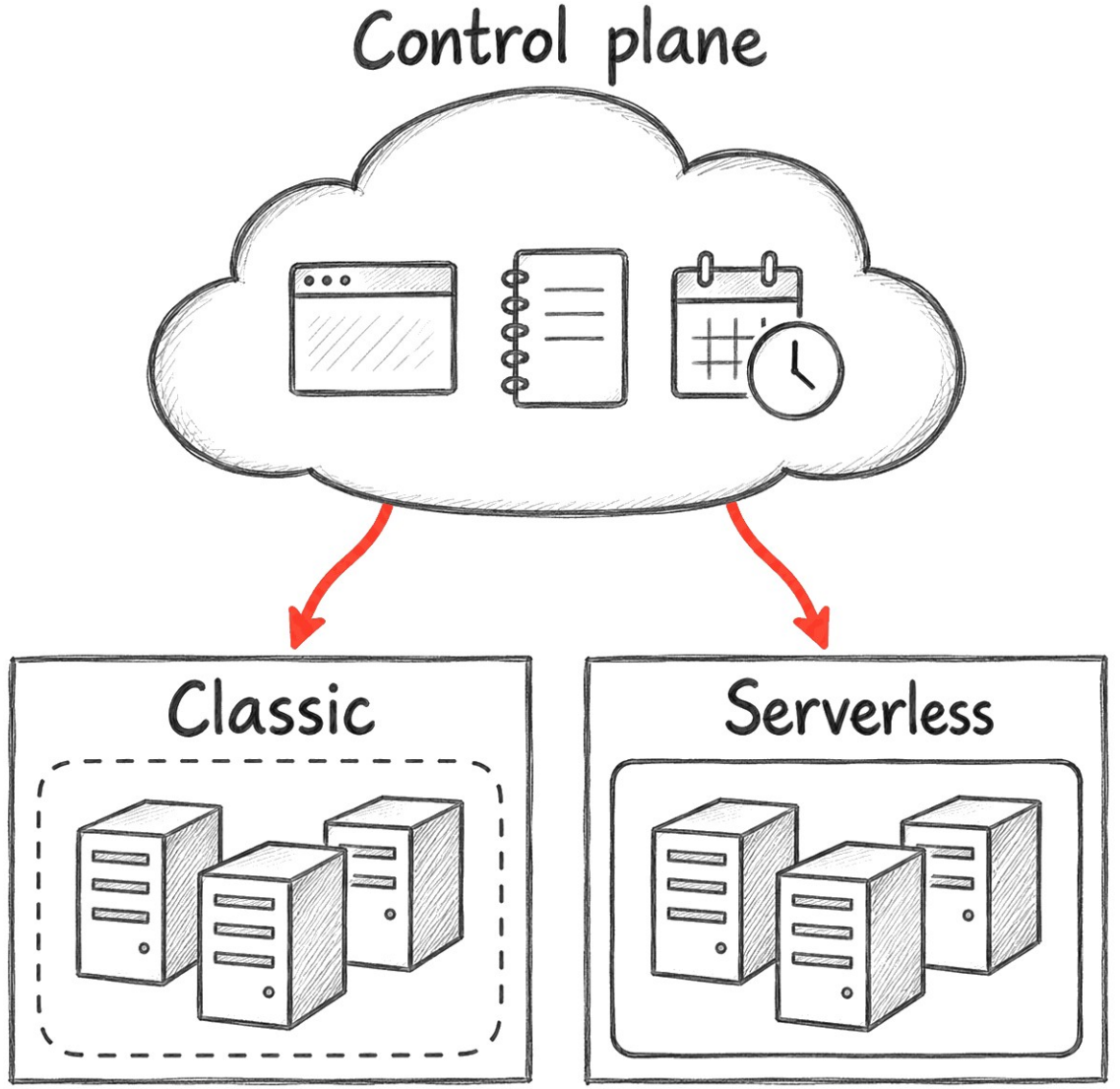
Tu wykonuje się kod.

Classic: zasoby klienta

Serverless: zasoby Databricks

Sieć zależy od typu compute.

Zatrzymanie compute nie usuwa danych w storage.



# Warstwy lakehouse

### Storage

Przechowuje pliki niezależnie od compute.

### Tabele

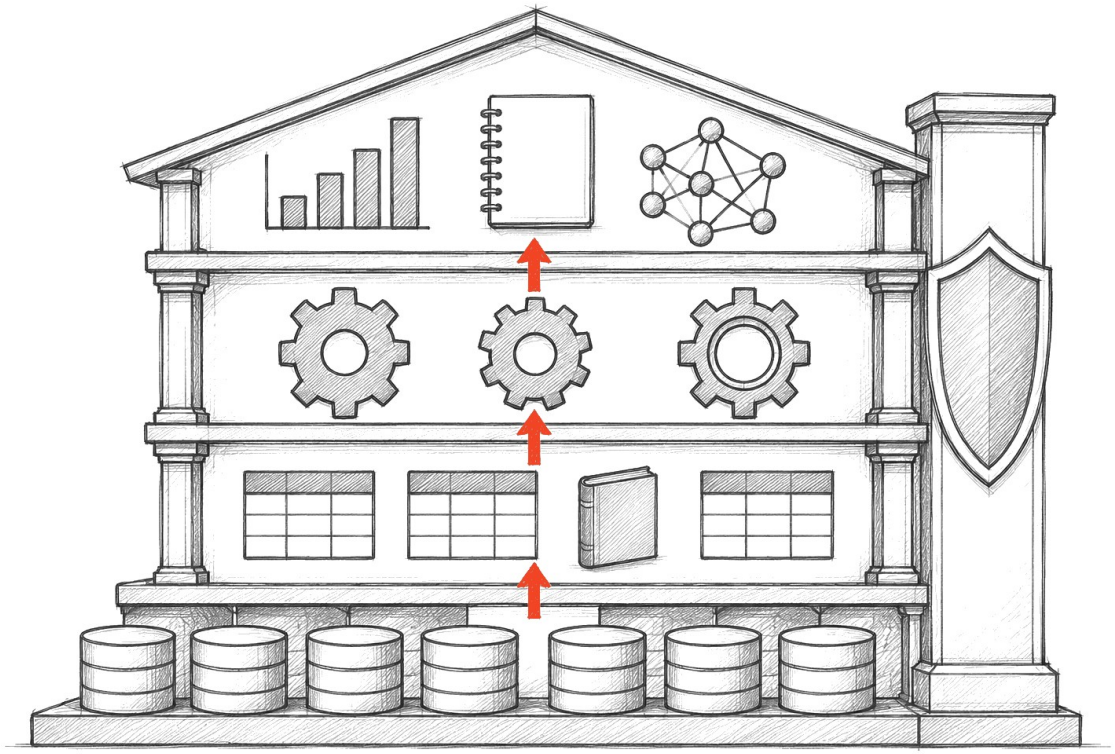
Delta Lake opisuje schemat, pliki i transakcje.

### Compute

Wykonuje kod notebooków, jobów i SQL.

### Governance

Unity Catalog kontroluje dostęp i metadane.



# Databricks w ekosystemie chmury

### Storage

Azure ADLS, Amazon S3, Google Cloud Storage

### Tożsamość

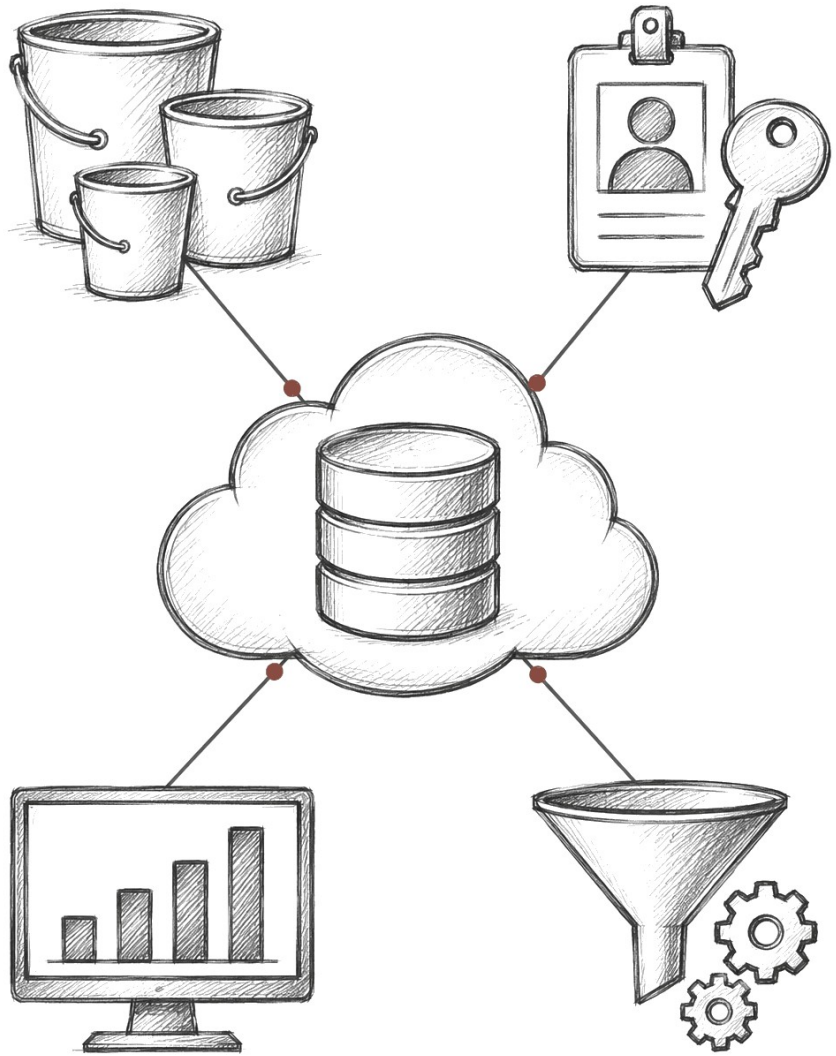
Entra ID: logowanie i grupy. Dostęp do storage konfigurujesz osobno.

### Narzędzia BI

Power BI, Tableau i inne narzędzia przez konektory oraz JDBC/ODBC.

### Integracje danych

Fivetran, dbt i Data Factory. Auto Loader do nowych plików.



# Co dzieje się po uruchomieniu kodu?



## Workspace

Tutaj piszesz i uruchamiasz kod.



## Compute

Wykonuje zapytanie lub transformację.



## Storage

Udostępnia pliki i przechowuje wynik.



## Unity Catalog

Sprawdza dostęp do obiektów danych.



## Delta Lake

Zapewnia spójny odczyt i zapis tabeli.

# Jak odnaleźć się w workspace

### Notebooki

Kod, opis i wyniki. Ćwiczymy w SQL i Pythonie.

### SQL editor i dashboardy

Zapytania na SQL warehouse i wizualizacja wyników.

### Catalog Explorer

Tabele, schematy, Volumes i uprawnienia.

### Git folders

Repozytorium, branch i commity. PR u dostawcy Git.

### Joby i pipeline’y

Jobs: kolejność zadań. Pipelines: deklaratywne przetwarzanie.

### Genie Code

Pomaga z kodem i błędami. Weryfikuj odpowiedzi.

# Katalog, schemat, obiekt

**Katalog**

Domena lub środowisko.

**Schemat**

Grupa powiązanych obiektów.

**Tabela · widok · volume**

Dane · zapytanie · pliki.

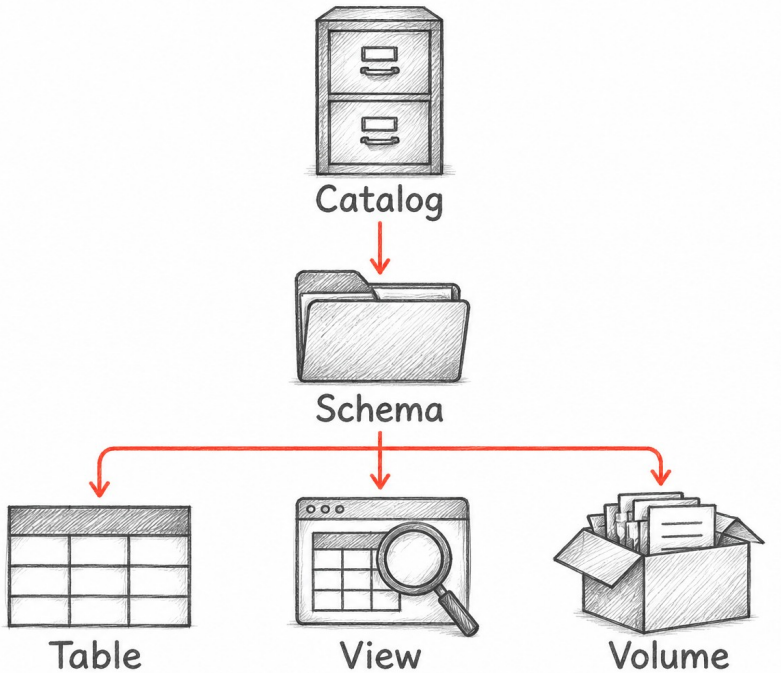



Tabela: <katalog>.bronze.lab_orders
Pliki: /Volumes/<katalog>/default/datasets/

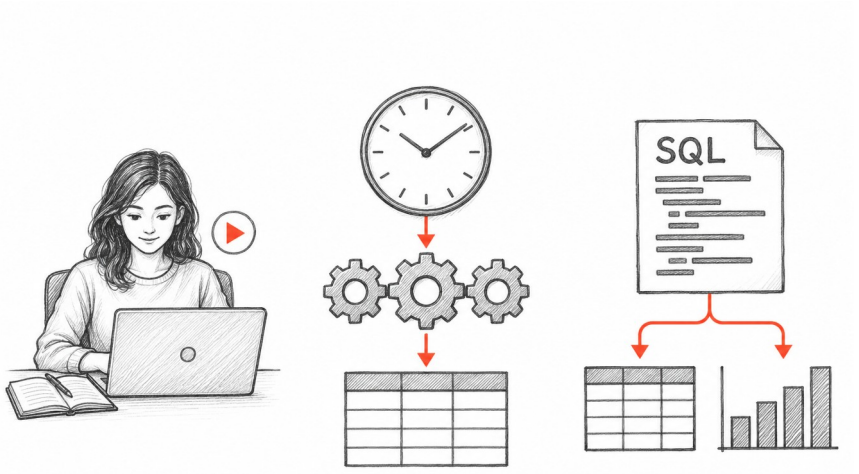
# Wybierz compute do zadania

## Classic czy serverless?

To wybór sposobu zarządzania zasobami.

## Najpierw zastosowanie

Notebook, job lub SQL/BI. Potem dobierz dostępny typ compute.




### Notebook

Interaktywna praca na serverless lub klasycznym compute.

### Job

Zaplanowane zadanie na serverless lub job compute.

### SQL i BI

SQL warehouse do zapytań i dashboardów.

# Kontroluj koszt obliczeń

Classic: DBU + maszyny. Serverless: usługa w DBU. Storage i transfer osobno.

### Dobierz compute

Inne potrzeby ma notebook, job i raport SQL.

### Zatrzymuj bezczynność

Ustaw auto-termination lub auto-stop.

### Ogranicz zasoby

Compute policies pomagają kontrolować konfigurację.

### Mierz zużycie

system.billing.usage pokazuje użycie. Koszt wymaga właściwej ceny DBU.

### Sprawdź przyczynę

Query Profile lub Spark UI przed zwiększeniem zasobów.

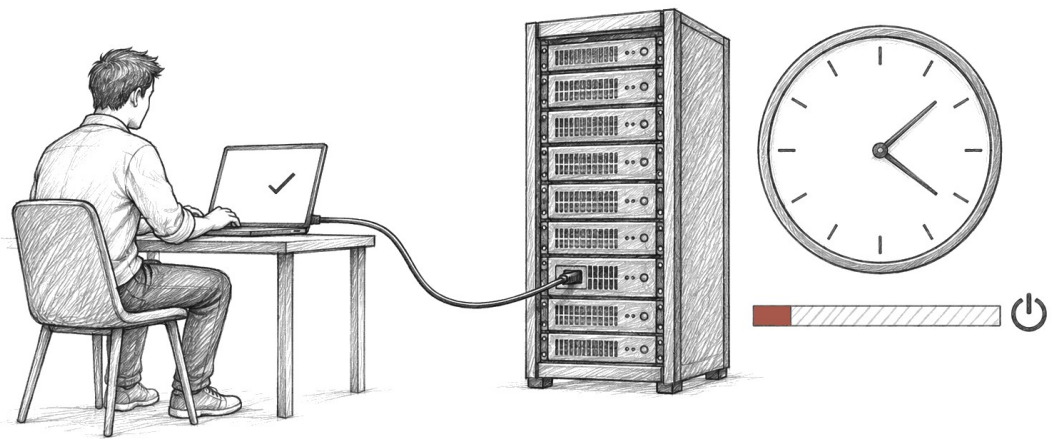
# Koszt bezczynności

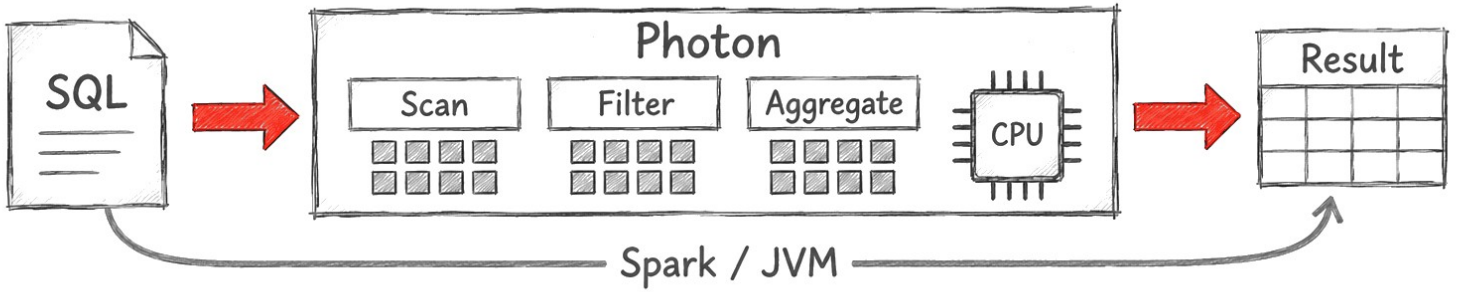
10 minut obliczeń

50 minut bezczynności

60 minut działania klastra

Co zmienisz przed kolejnym uruchomieniem?






# Czym jest Photon

Photon przyspiesza obsługiwane operacje SQL i DataFrame.

### Wektorowe wykonanie

Przetwarza dane kolumnowo na CPU.

### Zgodność

Nieobsługiwane operacje wykonuje Spark.

### Sprawdź efekt

Porównaj czas i koszt swojego zapytania.

M02 / APACHE SPARK

# Architektura Apache Spark

Driver, executory i wykonywanie obliczeń w SQL oraz PySpark.

# Driver, executory i partycje

### Driver

Planuje i koordynuje taski.

### collect()

Zbiera cały wynik do pamięci drivera. Duży wynik może jej nie zmieścić.

### Executory

Procesy wykonujące taski.

### Partycje

Fragmenty danych. W etapie jeden task przetwarza jedną partycję.

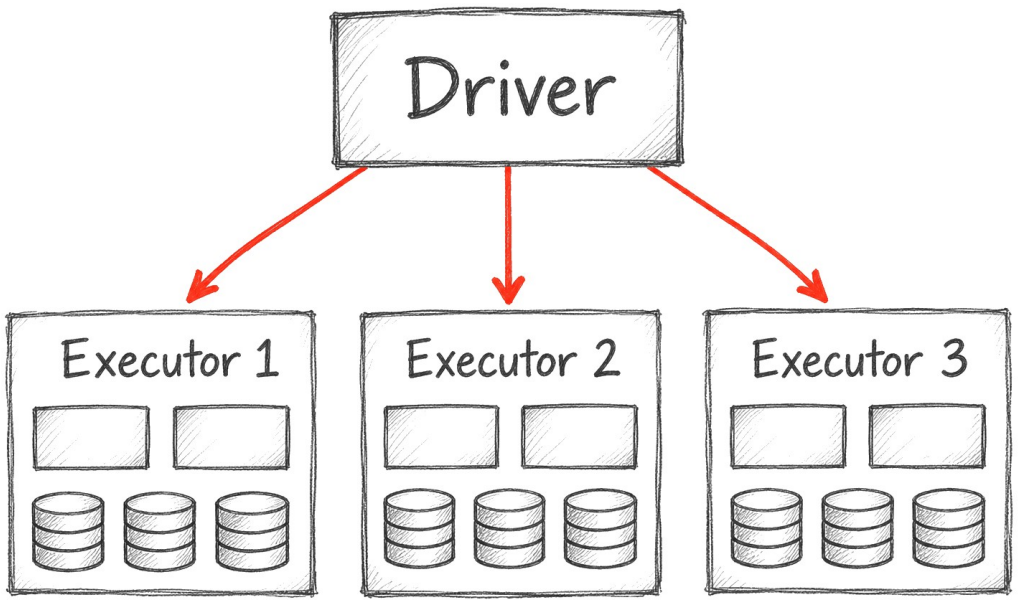
### Shuffle

Przenosi dane między partycjami. Może obciążać sieć i dysk.

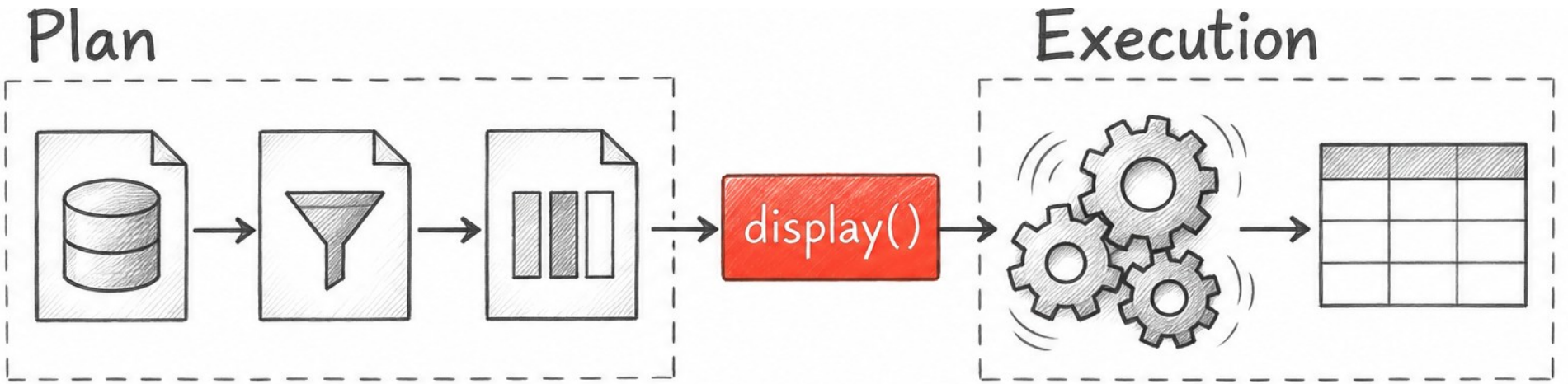
# Gdzie trafi wynik collect()?

Chcesz obejrzeć 10 wierszy dużej tabeli.

Czy collect() jest dobrym wyborem?
Co stanie się z wynikiem?



# Najpierw plan, potem obliczenie




**Transformacje budują plan**

select · filter · withColumn

**Akcje wykonują plan**

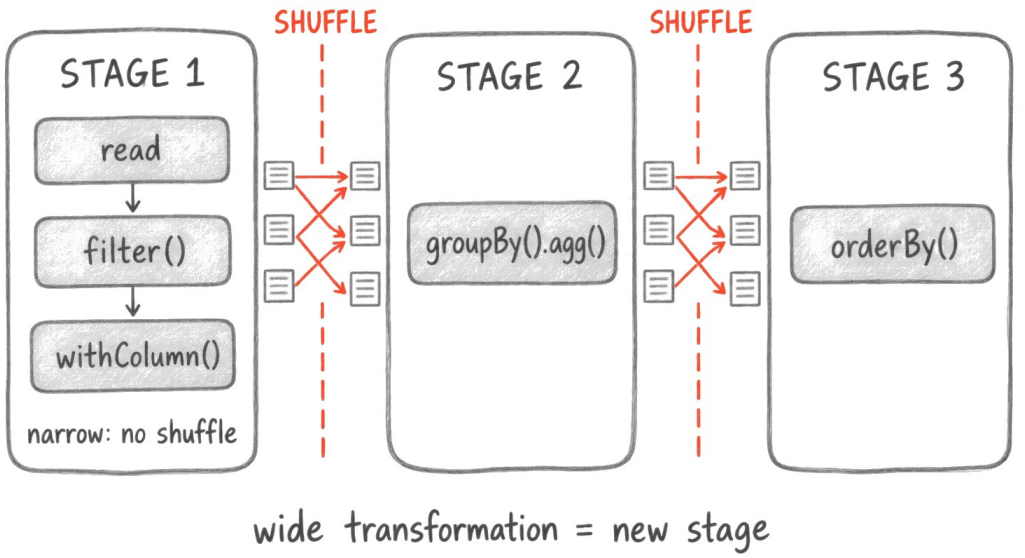
count · show · collect · zapis

**Która linia obliczy wynik?**

```
df2 = df.filter("amount > 0")
df2.count()
```

# DAG i plan wykonania

DAG opisuje zależności obliczeń.
Spark dzieli pracę na etapy i taski.
Shuffle przenosi dane między partycjami.
Optymalizator wybiera sposób wykonania.
Szczegóły planu: materiał dodatkowy.



# SQL i PySpark: dwa sposoby pracy

### DataFrame

Dane z nazwanymi kolumnami i schematem.

### Wyrażenie

Opisujesz obliczenie, bez pętli po wierszach.

### Podgląd wyniku

show i display. Sam plan nie zapisuje tabeli.

### Wspólny silnik

Spark optymalizuje SQL i operacje DataFrame.

M03 / INGESTIA

# Ingestia i medalion

Odczyt danych, transformacje i warstwy Bronze, Silver, Gold.

# Bronze, Silver, Gold

**Gold**

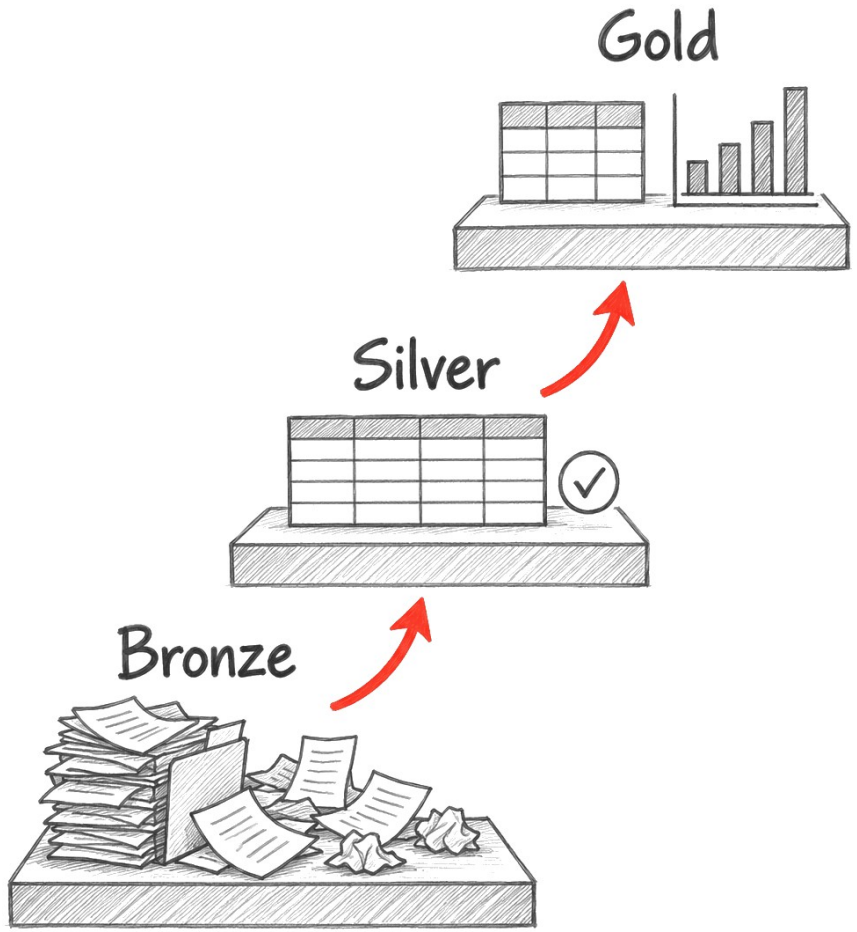
Wynik dla odbiorcy. Ustal znaczenie jednego wiersza.

**Silver**

Typy i jawne reguły jakości.

**Bronze**

Dane źródłowe i metadane odczytu.




Medalion to wzorzec. Reguły warstw implementujesz w kodzie.

# Jak wczytać dane?

Na szkoleniu zaczynamy od plików w Volume.

### Pliki w Volume

Odczyt: spark.read lub read_files().
Wynik: tabela Bronze.

### Auto Loader

Przyrostowy odczyt plików. Checkpoint przechowuje stan.

### Lakeflow Connect

Zarządzane konektory do baz i aplikacji.

### Streaming tables

Tabele zasilane zapytaniem strumieniowym.

# Warstwa Bronze, zasady

### Zachowaj wejście

Bronze pozwala ponowić transformacje.

### Dobierz typy

Tekst pomaga zachować błędne wartości. Nie jest wymogiem Delta Lake.

### Dodaj metadane

Czas odczytu i ścieżka pliku

### Ustal schemat

JSON ma typy, ale rekordy mogą się różnić.

### Zapisz do Delta

Transakcje i historia. Jakość sprawdzisz osobno.

# Schemat a jakość danych

| POLE | BRONZE → SILVER |
|---|---|
| order_id | STRING → STRING |
| total_amount | STRING → DOUBLE |
| order_datetime | STRING → TIMESTAMP |

### Schemat to struktura

Nie sprawdza znaczenia kwoty ani poprawności klucza.

### Rzutowanie to decyzja

Sprawdź wynik cast lub try_cast. Błędne wartości wymagają obsługi.

### NULL sprawdzaj jawnie

nullable=False w czytniku plików nie zastąpi walidacji.

# Silver: typy i reguły jakości

### 01 · Nadaj typy

Tekst zamień na liczbę lub datę.

### 02 · Uzgodnij reguły

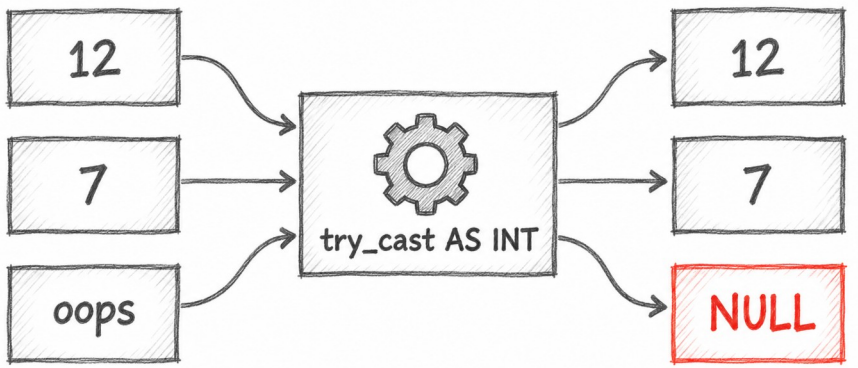
Co oznaczają pola i miary?

### 03 · Sprawdź jakość

Klucze, daty i duplikaty.

### 04 · Zapisz wynik

Zachowaj potrzebne metadane.




try_cast zwraca NULL przy błędzie. Zdecyduj, co zrobić z takim rekordem.

# Gold: jeden wiersz na dzień

## Ziarnistość

Co oznacza jeden wiersz wyniku?

## Miary

Jak obliczamy sumę, liczbę i średnią?

## Od szczegółów do agregatu

### Dwa zamówienia w jednym dniu

A: 100 zł  B: 200 zł

Wynik dzienny

Liczba: 2  Suma: 300 zł

Średnia: 150 zł

## Najpierw definicja miary

Czy kwota zawiera już rabat? W labie net_amount jest miarą ćwiczeniową.

# Warsztat 1 · Ingestia · 40 min

01

## Otwórz warsztat

WORKSHOP_ingestion + Workshop Guide

02

## Wykonaj kroki główne

01, 02, 03, 05, 07, 08, 09, 11

03

## Sprawdź krok

Uruchom kod, potem kliknij Check.

04

## Oceń dane

Typy, agregaty i metadane. PASS to początek kontroli.

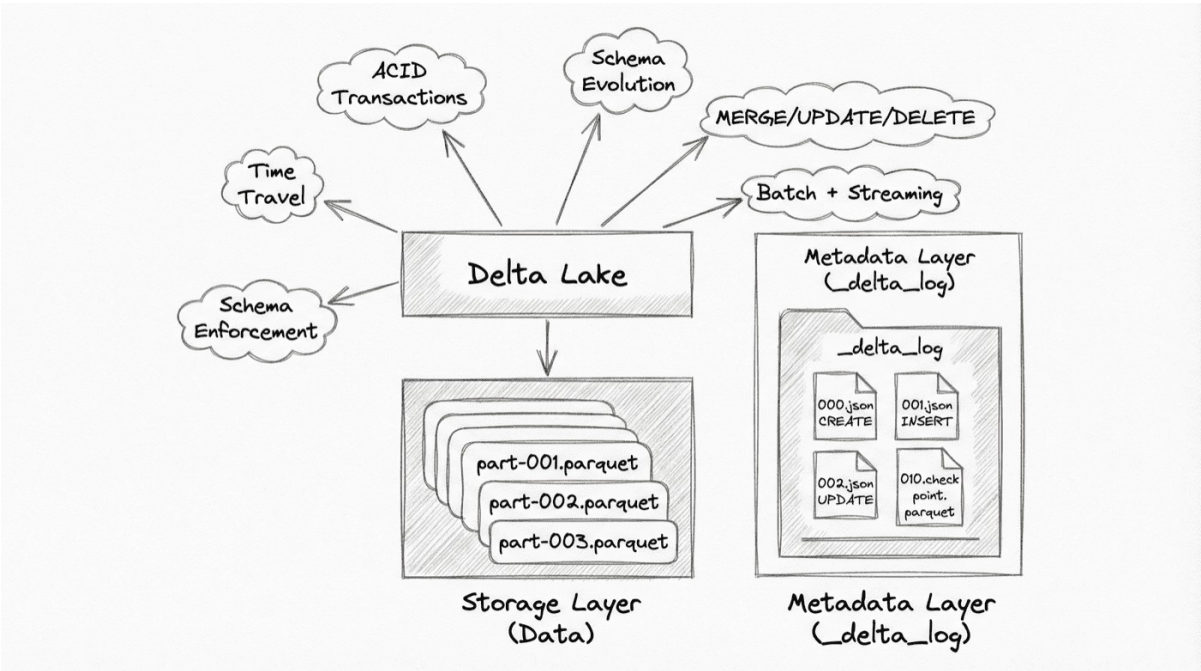
Wynik: Bronze lab_ingestion_customers z danymi klientów i metadanymi.

# Delta Lake

Transakcje, MERGE, historia danych i odzyskiwanie tabel.

# Czym jest Delta Lake?

Delta Lake = pliki Parquet + log transakcyjny
Atomowy zapis i spójny odczyt
Kontrola schematu
Wersje i historia danych
INSERT, UPDATE, DELETE, MERGE
Reguły biznesowe definiujesz osobno.



# Managed czy external?

Wybór dotyczy lokalizacji i cyklu życia plików.

## Managed

Databricks zarządza lokalizacją i cyklem życia danych.

Używamy w ćwiczeniach.

Utrzymanie może być automatyzowane.

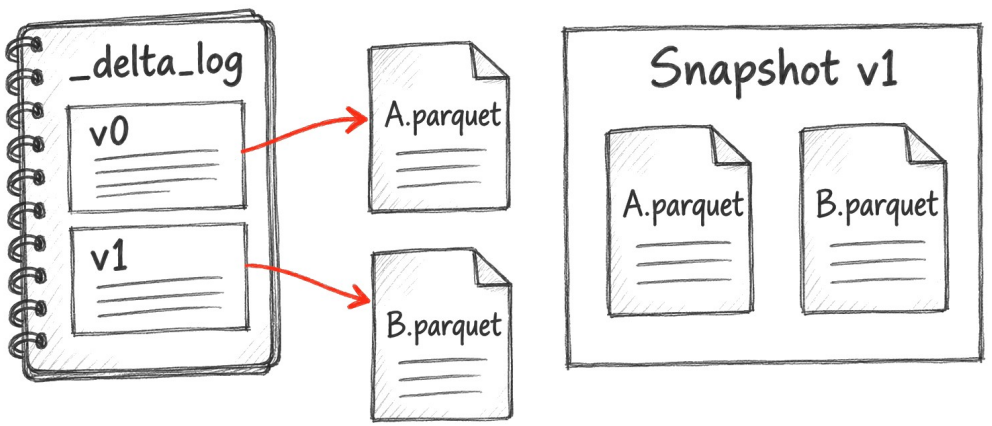
## External

Wskazujesz lokalizację danych.

DROP TABLE usuwa rejestrację, pozostawia pliki.

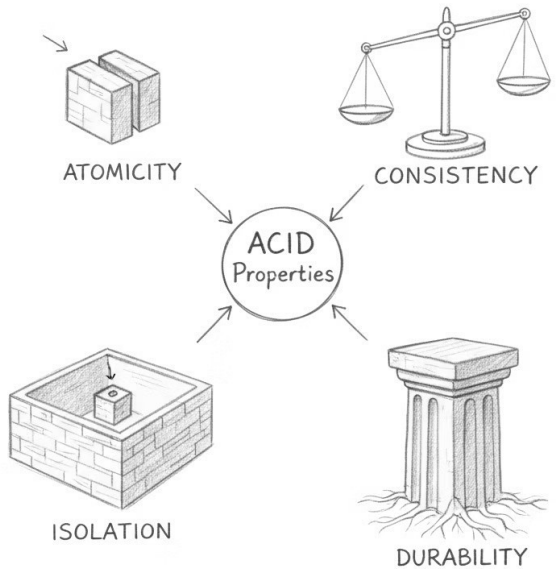
Zabezpiecz także dostęp do storage.

# Log tabeli Delta




_delta_log opisuje stan tabeli.
Dodane i wycofane pliki
Schemat i metadane
Informacje o transakcjach
Checkpointy pomagają odtworzyć stan.
Odczyt korzysta ze spójnego snapshotu.

# Własności ACID w data lake




## Atomowość

Cała transakcja jest widoczna po zatwierdzeniu.

## Spójność

Zapis respektuje schemat i jawne ograniczenia.

## Izolacja

Spójny odczyt. Zapisy mogą konfliktować.

## Trwałość

Zatwierdzone zmiany pozostają po awarii.

# MERGE: wybierz operację

```
DOPASUJ REKORD PO KLUCZU
Istnieje + zmiana → UPDATE
Istnieje + anulowanie → DELETE
Nie istnieje + aktywny → INSERT
Nie istnieje + anulowany → POMIŃ
```

### Bezpieczne ponowienie

Zależy od reguł i danych. Anulowany rekord nie może wrócić.

### Poprawny klucz

Określa, które wiersze opisują ten sam obiekt.

### Jednoznaczne źródło

Rozstrzygnij wiele zmian tego samego klucza.

# Schemat: kontrola i zmiana

### Kontrola zapisu

Niezgodny schemat może odrzucić zapis. Konwersje zależą od operacji.

### To nie walidacja biznesowa

Osobno sprawdź klucze i miary.

### mergeSchema

Dodaje kolumny w obsługiwanym zapisie. Uzgodnij zmianę.

### overwriteSchema

Zastępuje schemat przy overwrite. Użyj świadomie.

# Time travel

Podróż w czasie po historii danych

# Time travel: odczytaj starszy stan

## Historia zapisów i odczyt wersji

v0 Pierwszy zapis
v1 Dodanie danych
v2 Błędne usunięcie

Odczytujesz v1.
Bieżący stan nadal jest w v2.

Time travel tylko odczytuje
Nie zmienia bieżącego stanu.

### Wersja musi być dostępna

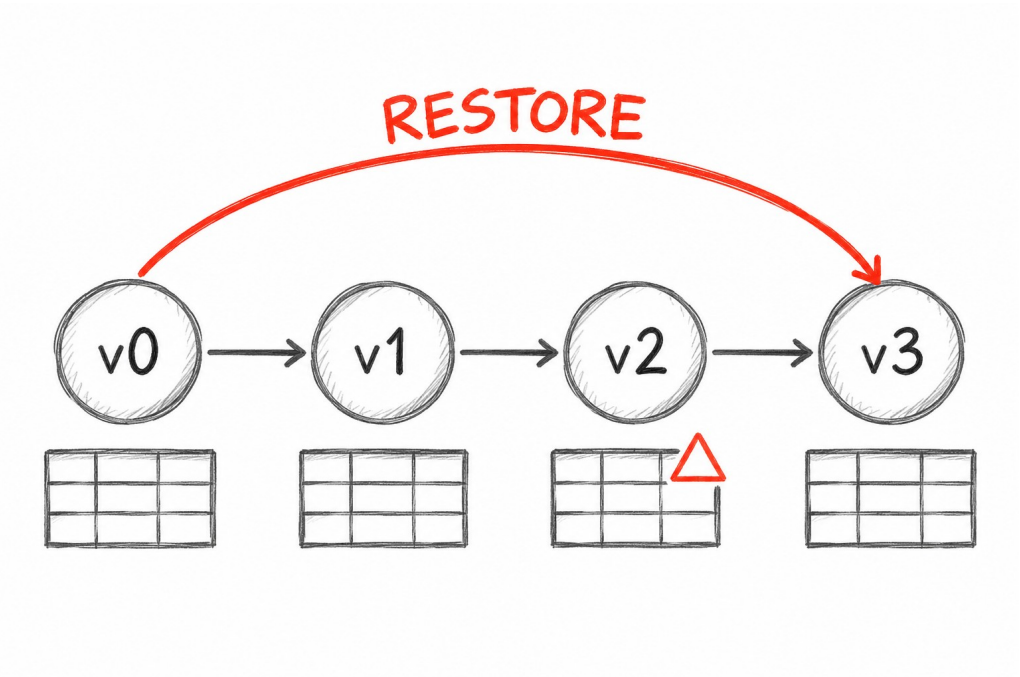
Potrzebne są pliki, log i dozwolona retencja.

### Przed RESTORE

Porównaj liczbę wierszy i agregaty. Ustal przyczynę błędu.

Retencja kosztuje storage.
Time travel to nie backup.

# RESTORE: przywróć stan tabeli




Przywracamy stan v0. Powstaje v3.
Wersje v1 i v2 pozostają w historii.

Historia pozostaje

RESTORE zapisuje nową wersję tabeli.

Sprawdź odbiorców

Operacja zmienia bieżący stan danych.

Pliki muszą istnieć

RESTORE nie odzyska plików usuniętych przez VACUUM.

# Utrzymanie tabel Delta

Poznaj cel operacji. Konfigurację dobierz do obciążenia.

### OPTIMIZE

Scala małe pliki w większe.

### VACUUM

Usuwa stare pliki po retencji. Ogranicza odzyskiwanie wersji.

### Liquid clustering

Układa dane pod zapytania. Alternatywa dla partycji i ZORDER.

### Automatyczne utrzymanie

Predictive optimization dla obsługiwanych tabel managed w UC. Compute jest płatny.

### Decyzja właściciela

Jak długo potrzebujesz odzyskiwać dane? Sprawdź retencję i koszt.

# Warsztat 2 · Delta Lake · 35 min

01

## Wykonaj kroki główne

01, 03, 05, 06, 08, 10, 12 w SQL

02

## Sprawdź wynik

Check w aplikacji oraz dane i historia.

03

## Napraw celowy błąd

Odczytaj starszą wersję. Użyj RESTORE.

04

## Wyjaśnij przyczynę

Schemat, klucz MERGE czy retencja?

Wynik: poprawny stan tabeli i operacja RESTORE w historii.

M05 / ORKIESTRACJA

# Lakeflow Jobs

Zależności notebooków, parametry, harmonogram i naprawa uruchomień.

# Po co nam orkiestracja?

### Ręczna kolejność

Łatwo pominąć krok lub użyć złych parametrów.

### Lakeflow Jobs

Zapisuje zależności i uruchamia taski według konfiguracji.

### Zależności i ponowienia

Kolejność, równoległość i retry per task.

### Compute dla joba

Serverless lub job compute. Kontroluj czas i koszt.

### Historia wykonań

Statusy, parametry, czas i logi tasków.

# Z czego składa się job?

## Task

Notebook albo zapytanie SQL.

## Depends on

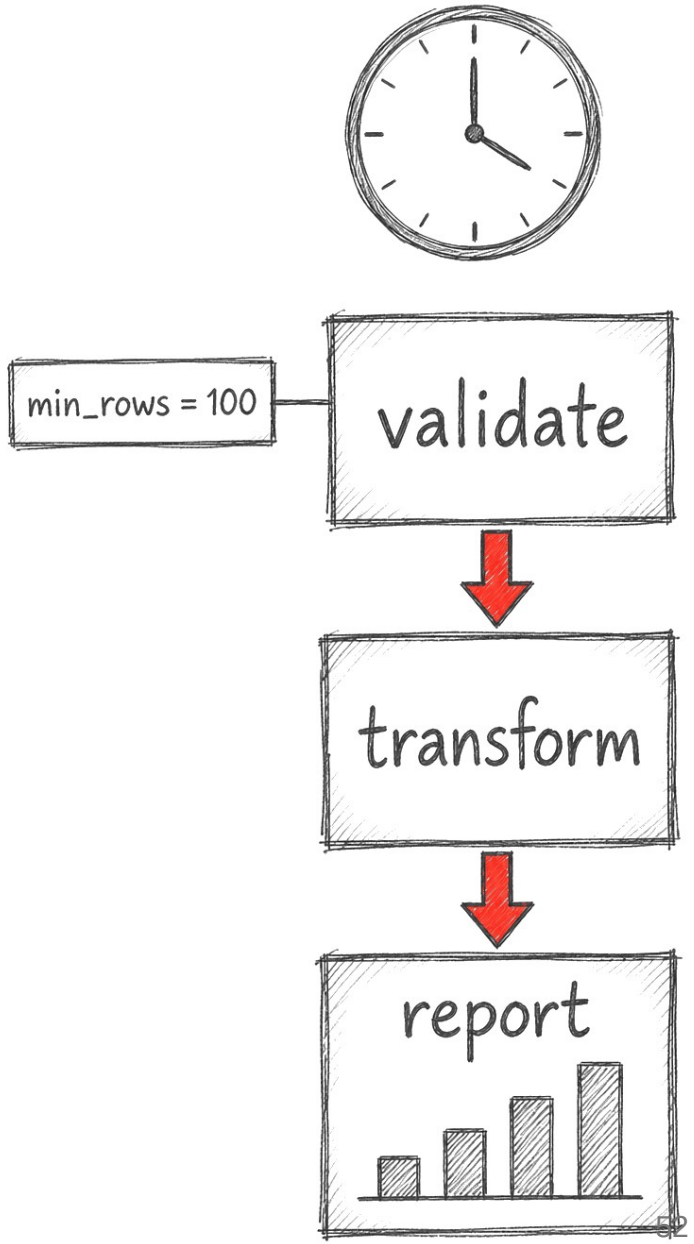
validate → transform → report

## Parametry

Wejście do taska; task values przekazują wynik.

## Ponowienia

Retry na błąd przejściowy. Kod najpierw napraw.



# Harmonogram i monitoring

Harmonogram

**0 0 6 * * ?**

06:00 w wybranej strefie

## Sposób uruchomienia

Ręcznie lub według harmonogramu

Po pojawieniu się pliku

Continuous: bieg po biegu

## Repair run

Napraw przyczynę, potem ponów wybrane taski.

## Alerty

Ustal, kto reaguje na błąd.

## Monitoring

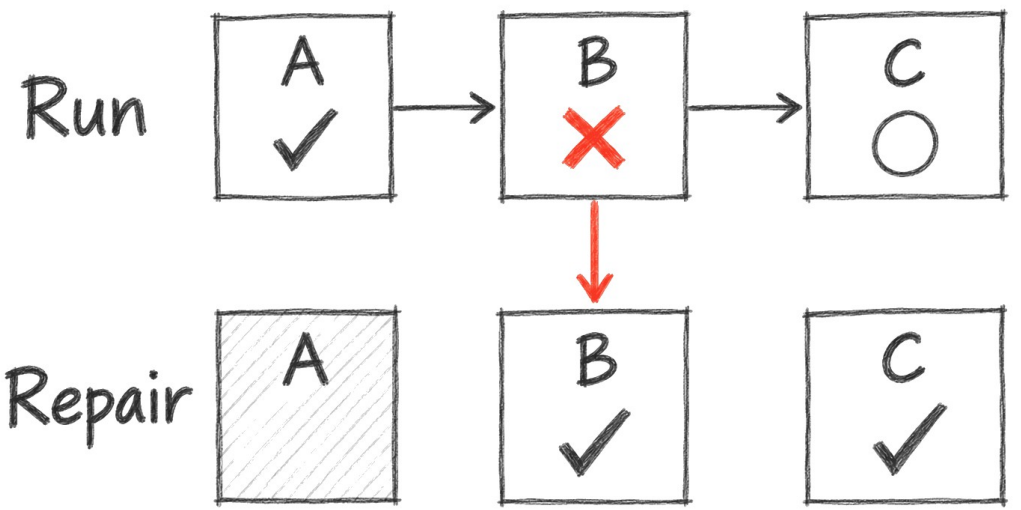
Sprawdź historię i logi.

W labie harmonogram: Paused.

# Co powtórzy Repair run?

A: sukces
B: błąd
C: nie uruchomiono

Które taski ponowisz po naprawie B?
Co trzeba sprawdzić przed ponowieniem?



M06 / GOVERNANCE

# Unity Catalog

Organizacja danych, uprawnienia i sprawdzanie dostępu.

# Unity Catalog w praktyce

### Metastore i workspace

Jeden metastore może obsługiwać wiele workspace w regionie.

### Lineage

Pokazuje pochodzenie danych dla obsługiwanych operacji.

### Audyt

Rejestruje zdarzenia dostępu i administracji.

### Tagi i komentarze

Pomagają opisać, znaleźć i sklasyfikować dane.

# Model uprawnień

```
USE CATALOG + USE SCHEMA + SELECT
GRANT: nadaj · SHOW GRANTS: sprawdź · REVOKE: odbierz
```

### Prawa są niezależne

Samo SELECT nie daje dostępu do katalogu i schematu.

### Użyj grupy kontowej

W ćwiczeniu: retailhub_analysts. Grupy lokalne workspace nie wystarczą.

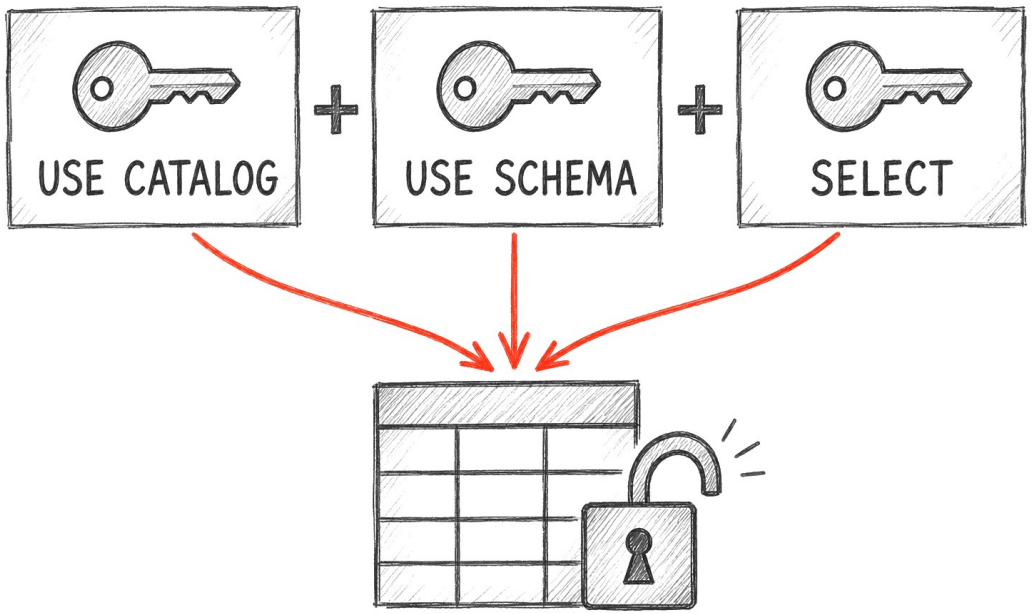
### Nadaj potrzebne prawa

Sprawdź dziedziczenie. Dostęp do compute kontrolujesz osobno.

# Prawa do odczytu

Użytkownik ma SELECT.
Zapytanie nadal nie działa.

Jakie uprawnienia sprawdzisz?
Czy dostęp do tabeli wystarczy?



PROJEKT KOŃCOWY

# Pipeline RetailHub

Łączymy Bronze, Silver i Gold w job. Sprawdzamy wynik i ponowienie.

# RetailHub: od zamówień do raportu

## Kontekst

Historyczne zamówienia w JSON.

## Zadanie

Dzienny agregat i job o 06:00.

W labie harmonogram: Paused.

## Wejście

100 000 rekordów z wielu dni.

## Wynik

gold.lab_daily_orders

Liczba, suma i średnia net_amount.

## Warunek

Ta sama paczka daje ten sam wynik.

# Architektura pipeline




Trzy notebooki i trzy tabele: Bronze → Silver → Gold.
Silver i Gold przeliczamy w całości.

# Gold: kolumny wyniku

| Kolumna | Typ | Opis |
|---|---|---|
| order_date | DateType | Dzień agregacji |
| order_count | LongType | Liczba wierszy w dniu |
| total_revenue | DoubleType | Suma net_amount z ćwiczenia |
| avg_order_value | DoubleType | Średnia net_amount, 2 miejsca |

net_amount to miara ćwiczeniowa. Kontrakt kwoty wymaga weryfikacji. Do rozliczeń dobierz DecimalType i reguły zaokrągleń.

# Projekt końcowy · 60 min

01

## Uzupełnij Silver i Gold

Uruchamiaj komórki CHECK.

02

## Kontroluj postęp

Po 25 min możesz użyć rozwiązań trenera.

03

## Zbuduj job

Bronze → Silver → Gold

04

## Uruchom i sprawdź

Kontrola w SQL. Harmonogram: Paused.

Wynik: 100 000 w Bronze · 90 904 w Silver · 32 dni w Gold

M07 / MODUŁ DODATKOWY

# AI/BI i Genie

Dashboardy i pytania do danych. Weryfikacja wyników oraz SQL.

# AI/BI i Genie, moduł dodatkowy

### Dashboard

Zapytania, wykresy i filtry.

### Odświeżanie

Na żądanie lub z harmonogramu. Możliwy cache.

### Przed publikacją

Sprawdź definicję miar i zakres danych.

### Genie Agents

Pytania w języku naturalnym na przygotowanych danych.

### Zweryfikuj odpowiedź

Sprawdź SQL oraz wynik.

### Udostępnianie

Dostęp zależy od trybu publikacji i uprawnień.

FINAŁ

# Podsumowanie

Kompetencje z modułów, pytania i dalsza nauka.

# Co warto zapamiętać

### Platforma

Compute liczy. Storage przechowuje dane.

### Medalion

Bronze: wejście. Silver: reguły. Gold: wynik dla odbiorcy.

### Delta

Transakcje i historia. Odzyskiwanie wymaga dostępnych plików.

### Schemat i jakość

Typy nie zastępują walidacji kluczy, wartości i miar.

### Jobs

Zapisz zależności. Sprawdź bezpieczne ponowienie.

### Unity Catalog

USE CATALOG + USE SCHEMA + SELECT. Compute osobno.

# Co dalej

### Powtórz ćwiczenia

Użyj Free Edition. Sprawdź jej aktualne limity.

### Zmień dane wejściowe

Dodaj kolumnę, wywołaj błąd i przywróć poprawny stan.

### Zaplanuj certyfikację

Sprawdź zakres Data Engineer Associate.

### Wybierz dalszy temat

Streaming, governance lub wydajność.

### Korzystaj z dokumentacji

docs.databricks.com i release notes

### Weryfikuj odpowiedzi AI

Uruchom kod i sprawdź wynik oraz źródła.

# Pytania i feedback

## Co potrafisz wyjaśnić?

Dlaczego collect() może przeciążyć driver?

Dlaczego SELECT może nie wystarczyć?

Kiedy time travel nie odzyska danych?

## Ankieta

Co pomogło Ci zrozumieć platformę?

Co wymaga lepszego wyjaśnienia?

## Kontakt

Dane prowadzącego są na końcu ankiety.

Wybierz jeden moduł do samodzielnego powtórzenia.